npj | computational materials

Article

Published in partnership with the Shanghai Institute of Ceramics of the Chinese Academy of Sciences

https://doi.org/10.1038/s41524-024-01378-4

Learning dislocation dynamics mobility laws from large-scale MD simulations

Check for updates

Nicolas Bertin ✉, Vasily V. Bulatov & Fei Zhou

By dispensing with all the atoms and only focusing on dislocation lines, the computational method of Discrete Dislocation Dynamics (DDD) gains greatly over Molecular Dynamics (MD) in simulation efficiency of metal plasticity. But whereas in MD dislocations follow natural dynamics of atomic motion, DDD must rely on a dislocation mobility function to prescribe how a dislocation line should respond to the driving force exerted on it. However, reflecting our still incomplete understanding of ways in which dislocations move, mobility functions presently employed in DDD simulations entail simplifications and approximations of limited or, worse still, unknown accuracy and applicability. Here we introduce a data-driven approach in which the dislocation mobility function is modeled as a graph neural network (GNN) trained on large-scale MD simulations of crystal plasticity. We apply our proposed approach to predicting plastic strength of body-centered-cubic (BCC) metal tungsten and show that, once implemented in a DDD model, our GNN dislocation mobility function accurately reproduces the challenging tension/compression asymmetry of plastic flow observed both in ground-truth MD simulations and in experiment. Furthermore, subsequently validated by MD simulations, the same function accurately predicts plastic response of tungsten under conditions not previously seen in training. By demonstrating its ability to learn relevant physics of dislocation motion, our DDD+ML approach opens a promising avenue to bringing fidelity of DDD models closer in line with direct MD simulations at a much reduced computational cost.

The mesoscopic method of Discrete Dislocation Dynamics (DDD) aims to compute crystal plasticity response from the motion and interaction of dislocations[1–7]. DDD is defined on dislocation lines represented as interconnected dislocation segments thus reducing the number of degrees of freedom by many orders of magnitude compared to an atomistic model of the same crystal. Given its relative computational efficiency, DDD has been long regarded as a promising scale-bridging method capable of directly connecting macroscopic crystal plasticity to the underlying microscopic mechanisms of dislocation motion. It is only relatively recently, owing to the steadily growing HPC capabilities, that fully atomistic Molecular Dynamics (MD) simulations of bulk crystal plasticity have become possible[8–11]. Still, owing to their high computational cost, MD simulations of crystal plasticity remain limited to deformation rates many orders higher than rates typical of quasistatic mechanical tests. Furthermore, MD simulations are presently limited to sub-micron length scales that are too small compared to length scales characteristic of dislocation microstructures observed under quasistatic deformation conditions. Thus, when it comes to quasistatic deformation, DDD is likely to remain the method of choice for predicting crystal plasticity from dislocation motion for years to come.

The key material-specific ingredient of a DDD model encapsulating the physics of dislocation motion is a dislocation mobility function used to predict velocity of a dislocation segment in response to mechanical force or stress seen by the segment. Typically, dislocation motion is assumed to be over-damped (ignoring inertia) and its velocity is computed as

$$\boldsymbol{V}_i = \frac{d\boldsymbol{r}_i}{dt} = \mathcal{M}\left[\boldsymbol{F}_i\left(\{\boldsymbol{r}_j\}, \{\boldsymbol{b}_{jk}\}\right)\right] \tag{1}$$

where $\boldsymbol{r}_j$ and $\boldsymbol{b}_{jk}$ are positions of nodes and the Burgers vectors of segments defining the set of vertices and edges of a geometric graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ representing the dislocation network.

Physical fidelity of a DDD simulation is predicated on availability of an accurate dislocation mobility function $\mathcal{M}$. Owing to difficulties in extracting the force-velocity relationship from experiments, dislocation theory has been used to predict dislocation mobility, e.g. by computing the nucleation energy barrier and the critical stress for dislocation motion controlled by kink-pair mechanisms[12–14] or by extracting a dislocation drag coefficient from the spectrum of thermal fluctuations[15]. It has become common practice to calibrate dislocation mobility functions against MD simulations

Lawrence Livermore National Laboratory, Livermore, CA, USA. ✉e-mail: bertin1@llnl.gov

of individual dislocations. Within this approach, a parametric functional form of the mobility function is "hand-crafted" based on physical knowledge or intuition and its parameters are fitted to reproduce dislocation velocities observed in MD calculations of isolated dislocations of various line orientations (e.g. edge, screw, mixed) performed under stress and temperature conditions of interest[14,16–22]. Widely adopted, the conventional calibration workflow is based on a rarely articulated assumption of *transferability*[23], namely that mobility of individual/isolated dislocations is representative of mobility of dislocation segments incorporated in a dislocation networks formed under deformation. Such an assumption leaves aside possible effects of dislocation *connectedness* on dislocation mobility, such as was recently observed in ref. 24.

Here we propose an alternative approach to calibration of dislocation mobility functions that does not rely on the assumption of transferability and accounts for network effects on dislocation motion. Instead of extracting mobility parameters from MD simulations of individual dislocations, in our proposed DDD+ML approach dislocation mobility is predicted by a graph neural network (GNN) trained on dislocation trajectories extracted from massive MD simulations of crystal plasticity. In training our GNN-based mobility functions we regard MD simulations as the ground-truth that subsequent DDD+ML simulations are intended to reproduce. Using body-centered cubic (BCC) tungsten as a testbed material model, we show that GNN can learn non-trivial physics of dislocation motion and, when subsequently used as a mobility function in DDD simulations, matches plasticity response observed in MD simulations used in training as well as in MD simulations performed under previously unseen deformation conditions.

Results

DDD+ML model

Viewed as a network, dislocation microstructure is a natural geometric graph (Fig. 1). Recently explored in a simplified model[25], this correspondence is further exploited here by training a GNN to predict mobility of dislocation segments comprising dislocation networks. For training the GNN we use dislocation network configurations extracted using the dislocation extraction algorithm (DXA)[26,27] from large-scale (ground-truth) MD simulations.

In a nodal version of DDD widely adopted by its practitioners, training a GNN to serve as a dislocation mobility function entails learning a relationship between the input (force on the node) and the output (node velocity) vectors given the instantaneous geometry and topology of the dislocation network (Eq. (1)). Although the task may appear straightforward, two practical complications arise. First, it is difficult to accurately define mechanical force acting on a dislocation node in an atomistic MD model (more on this later). Second, the DXA algorithm extracts only instantaneous configurations of dislocation networks thus velocities of dislocation nodes have to be inferred from displacements of dislocation nodes over the time interval between two subsequent DXA snapshots. Consider two graphs $\mathcal{G}^t$ and $\mathcal{G}^{t+\Delta t}$ representing consecutive states of a dislocation network at times t and $t+\Delta t$, respectively (Fig. 1a). More often than not, graphs $\mathcal{G}^t$ and $\mathcal{G}^{t+\Delta t}$ constructed by the DXA algorithm are not topologically isomorphic, meaning in particular that some of the graph nodes present at time t no longer exist at time $t+\Delta t$ and vice versa. This complication is due to different meshing of dislocation lines in two DXA networks as well as due to topological events (e.g. dislocation collisions, cutting, etc) taking place over the time interval between t and $t+\Delta t$. This

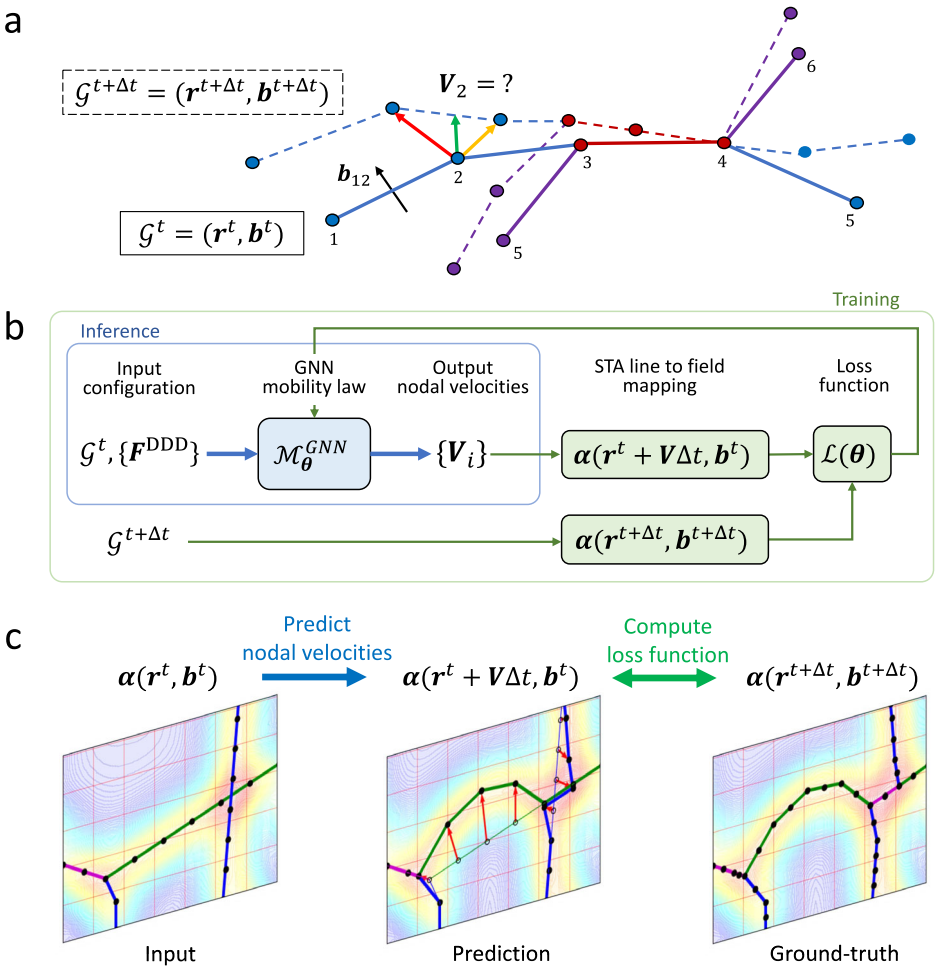

Fig. 1 | The DDD+ML approach. a A schematic showing two consecutive dislocation networks $\mathcal{G}^t$ and $\mathcal{G}^{t+\Delta t}$ extracted using DXA from two MD trajectory snapshots at times t (solid lines) and $t+\Delta t$ (dashed lines). As returned by DXA, two network configurations are defined by nodal positions $\boldsymbol{r} = \{\boldsymbol{r}_i\}$ and Burgers vectors of their dislocation segments $\boldsymbol{b} = \{\boldsymbol{b}_{ij}\}$, but are generally non-isomorphic and contain no information about mechanical forces driving dislocation motion. Not having the same nodes present in both networks leaves it uncertain how to define nodal velocities. For instance, three arrows of different colors pointing out of node 2 show three possible ways to define its velocity $\boldsymbol{V}_2$. **b** A schematic of inference and training loops of our proposed workflow for developing a GNN dislocation mobility function. **c** An illustration of the Nye's tensor field-matching approach circumventing the ill-defined problem of matching nodal velocities. As exemplified here, our field-matching procedure is agnostic to details of line discretization and network topology.

lack of isomorphism makes it impossible to establish a one-to-one correspondence between dislocation nodes in two consecutive network configurations as needed to define nodal velocities. As such, learning on nodal velocity vectors is not a well-defined task for machine-learning (ML).

We propose to address both these challenges of unknown ground-truth force and velocity vectors in the following way. First, to address the issue of defining the mechanical force $\boldsymbol{F}_i$ on a dislocation node i, we assume that a substantial part of it seen by each node in an MD simulation can be estimated from the standard solutions of dislocation theory commonly employed in DDD simulations, e.g.

$$\boldsymbol{F}_i^{\mathrm{DDD}} = \boldsymbol{F}_i^{\mathrm{ext}} + \boldsymbol{F}_i^{\mathrm{int}} + \boldsymbol{F}_i^{\mathrm{core}} \tag{2}$$

where $\boldsymbol{F}_i^{\mathrm{ext}}$ is mechanical force due to an externally applied stress, $\boldsymbol{F}_i^{\mathrm{int}}$ is due to elastic interactions among dislocation segments, and $\boldsymbol{F}_i^{\mathrm{core}}$ is associated with the dislocation core. That this DDD force is only an approximation of the true force $\boldsymbol{F}_i^{\mathrm{MD}}$ exerted on node i in an MD simulation can be expressed as

$$\boldsymbol{F}_i^{\mathrm{MD}} = \boldsymbol{F}_i^{\mathrm{DDD}} + \boldsymbol{F}_i^{\mathrm{corr}} \tag{3}$$

where $\boldsymbol{F}_i^{\mathrm{corr}}$ is an unknown correction term accounting for the error made in approximating the true force using Eq. (2). Given that the external and the interaction terms in the DDD force are smoothly varying functions that are accurately and efficiently computed in DDD, the correction term in Eq. (3) is expected to be a "shortsighted" function of the local network topology and geometry which we leave for the GNN model to learn. Thus we define the nodal mobility function $\mathcal{M}_{\boldsymbol{\theta}}^{\mathrm{GNN}}$ as a parametric function with learnable parameters $\boldsymbol{\theta}$,

$$\boldsymbol{V}_i = \mathcal{M}_{\boldsymbol{\theta}}^{\mathrm{GNN}}\left(\boldsymbol{F}_i^{\mathrm{DDD}}, \{N_j\}, \{E_{jk}\}\right), \tag{4}$$

and let the GNN model learn the resulting nodal velocity vector $\boldsymbol{V}_i$ as a function of the estimated force $\boldsymbol{F}_i^{\mathrm{DDD}}$ and a local graph neighborhood of node i with nodal and edge attributes N_j and E_{jk} (see Methods for a detailed definition of the local graph and its attributes). So-defined and properly trained, the mobility function in Eq. (4) is expected to learn the unknown but local correction to the nodal force $\boldsymbol{F}_i^{\mathrm{corr}}$. To the extent that the correction term is indeed learnable from the ground-truth MD data and considering that nodal forces in DDD are always computed using Eq. (2), our formulation should ensure accuracy of subsequent rollout DDD simulations.

To address uncertainty of defining nodal velocity vectors in the ground-truth MD data we resort to a recently proposed sweep-tracing algorithm (STA)[10] that deals with precisely the same uncertainty. STA employs a dual line/field representation of a dislocation network to "reconnect" any two successive dislocation networks even when they are not topologically isomorphic. In STA, Nye's tensor fields produced by two dislocation networks are first computed and mapped on a static spatial grid. The task of reconnection is then defined as an optimization problem that seeks to minimize a properly defined distance between Nye's tensor[28] field representations of two networks[10].

Borrowing from the STA approach, training of the GNN-based mobility function $\mathcal{M}_{\boldsymbol{\theta}}^{\mathrm{GNN}}$ in Eq. (4) is achieved by minimizing over parameters $\boldsymbol{\theta}$ the following loss function

$$\mathcal{L}(\boldsymbol{\theta}) = \sum_s \sum_{\boldsymbol{g}} \sum_{kl} \left[\alpha_{kl}^{\boldsymbol{g}}\left(\boldsymbol{r}^{t_s} + \mathcal{M}_{\boldsymbol{\theta}}^{\mathrm{GNN}}\Delta t_s, \boldsymbol{b}^{t_s}\right) - \alpha_{kl}^{\boldsymbol{g}}\left(\boldsymbol{r}^{t_s+\Delta t_s}, \boldsymbol{b}^{t_s+\Delta t_s}\right)\right]^2 \tag{5}$$

where the sums are over the set of training examples s, grid points $\boldsymbol{g}$ and Cartesian components of the Nye's tensor field computed on the grid points using the method introduced in[29]. During training, the GNN learns to predict nodal velocity vectors $\boldsymbol{V}_i$ at time t_s, by best matching its predictions to the field representation of the network at a later time $t_s + \Delta t_s$, Fig. 1c. Defining and computing training loss on a static grid addresses the seemingly ill-defined problem of matching topology of two generally non-isomorphic graphs.

Calculations of the Nye's tensor field produced by dislocation networks are only required for computing and back-propagating parameters $\boldsymbol{\theta}$ during training. Once trained, only the GNN-based mobility function in Eq. (4) needs to be evaluated at the inference time, Fig. 1b. Our GNN model and its training protocol are implemented in PyTorch[30].

Validation on DDD trajectories

Although our intent is to train GNN-based mobility functions on dislocation trajectories extracted from MD simulations, it can be insightful to first test the method by observing if and how well a GNN trained on a DDD simulation itself can reproduce its known (user-defined) mobility function. For this purpose here we use a generic linear function previously developed for BCC metals[31] and implemented in our DDD code ParaDiS[7]

$$\boldsymbol{V}_i = \mathcal{M}^{\mathrm{DDD}}(M_e, M_s) \cdot \boldsymbol{F}_i^{\mathrm{DDD}} \tag{6}$$

where $\mathcal{M}^{\mathrm{DDD}}$ is a mobility tensor parameterized by two scalar coefficients $M_e = 2600(\mathrm{Pa{\cdot}s})^{-1}$ and $M_s = 20(\mathrm{Pa{\cdot}s})^{-1}$ defining mobility of an isolated edge and an isolated screw dislocation, respectively.

To generate training data, we ran five DDD simulations starting from five different initial distributions of 12 prismatic loops seeded at random positions. Identical except for their initial states, five crystals were subjected to compressive straining along their [001] axes at a constant strain rate of 2×10^8/s which are also the conditions of MD simulations intended for training (see next section). The five simulations were run to 0.5 ns over which time dislocation networks were saved at regular intervals of 0.25 ps thus producing 10,000 configurations for training. Following the procedure described in "DDD+ML model", for each training configuration the DDD nodal forces and Nye's tensor fields were computed and the loss (Nye's tensor mismatch, Eq. (5)) was minimized by back propagation.

Once trained, the resulting GNN-based mobility function can be used to infer nodal velocities in any dislocation configuration, including two simple cases of infinite edge and screw dislocations moving in response to applied shear stress τ. Such simple exercise allows us to pick into the otherwise "black-box" trained mobility function. As shown in Fig. 2a, the linear mobility employed in the DDD simulations is well reproduced by the learned GNN mobility function. Linear regression on predicted stress-velocity curves yields mobility coefficients of $\hat{M}_e \approx 2564\,(\mathrm{Pa}\cdot\mathrm{s})^{-1}$ and $\hat{M}_s \approx 6\,(\mathrm{Pa}\cdot\mathrm{s})^{-1}$ for the edge and screw dislocations, respectively, in agreement with our-defined mobility parameters for pure screw and pure edge dislocations. A notable discrepancy in the screw mobility is likely caused by relative scarcity in the training data of instances in which screw dislocations actually move. Under assumed extreme anisotropy of dislocation mobility (ratio $M_e/M_s = 23$) and initiated from 12 prismatic loops of pure edge character, it is mostly edge and mixed dislocations that move over the first 0.1 of strain covered by five DDD simulations while the screws are passively drawn while remaining largely motionless. Training on more extended DDD simulations reaching strains at which screws become more active, is likely to bring the learned $\mathcal{M}_{\boldsymbol{\theta}}^{\mathrm{GNN}}$ in closer agreement with the user-defined mobility parameter for screw dislocations.

Training on MD trajectories

We now apply our method to learning a dislocation mobility function on data from large-scale MD simulations. To illustrate the approach, here we focus on plasticity in body-centered-cubic (BCC) metals challenging for its strong plastic anisotropy[32–34] manifested in the notorious tension/compression asymmetry[10,35,36] and rather uncommon slip crystallography[37]. We choose BCC tungsten as a testbed material because it is nearly elastically isotropic making it particularly convenient for comparisons with DDD simulations which are simpler and much more expedient to perform in an elastically isotropic crystal. For MD simulations here we employ a

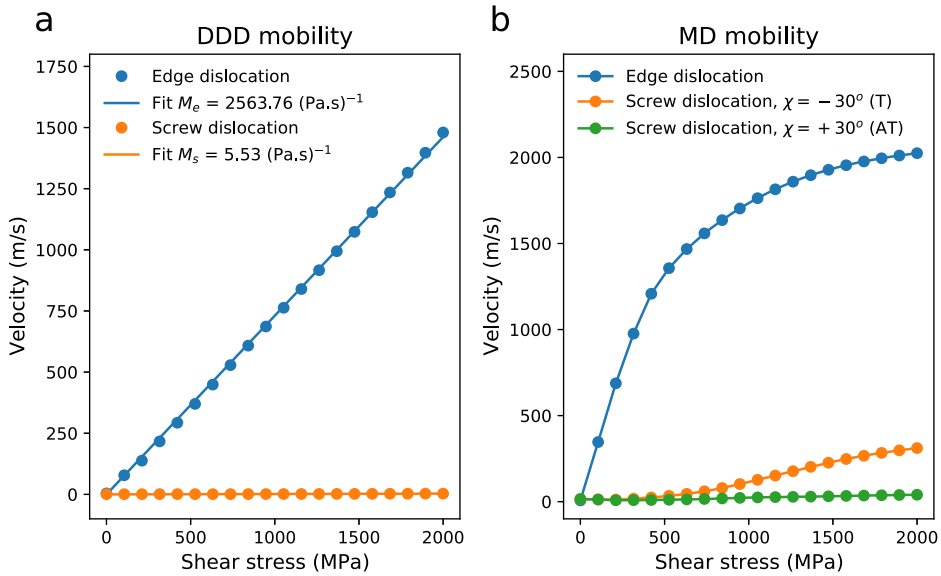

Fig. 2 | Stress-velocity predictions. a Velocities of isolated edge and screw dislocations predicted using a GNN mobility function trained on DDD trajectories. The learned GNN model accurately reproduces linear mobilities employed in DDD simulation. **b** Velocities of isolated edge and screw dislocations predicted using a GNN mobility function trained on large-scale MD trajectories of BCC tungsten deformed under [001] tension and compression. The trained GNN mobility function predicts a non-linear force-velocity relation for the edge dislocation and a strong asymmetry in the velocities of screw dislocations sheared in the twinning (T) and anti-twinning (AT) direction.

computationally efficient inter-atomic potential (IAP) of the embedded-atom (EAM) type previously developed for tungsten[38] with its Zener anisotropy ratio computed at 300 K close to 1.0 at $A = 2C_{44}/(C_{11} - C_{12}) = 1.15$.

We first perform two MD simulations with ~35 millions atoms, much the same way as in the DDD validation simulations described earlier. BCC crystals are initially seeded with prismatic dislocation loops and then deformed at 300 K under uniaxial tension and compression along the [001] direction. Again consistent with the earlier DDD simulations, the crystals are deformed at a true strain rate of 2×10^8/s until reaching a true strain of 1.0. Consistent with our previous results for BCC tantalum[10,39], the simulations predict a well defined tension/compression asymmetry, with flow stress of ~4.2 GPa under compression and ~2.8 GPa under tension. During both runs, DXA is used to extract dislocation configurations at equal time intervals of 1 ps thus producing a total of 10,000 DXA snapshots subsequently used to train a GNN-based mobility function as previously described.

When initially tested in predicting velocities of isolated edge and screw dislocations, the model exhibits smooth and well-behaved functions, Fig. 2b. Velocity predicted for an edge dislocation is linearly proportional to stress up to ~300 MPa, followed by a non-linear regime reaching an asymptotic velocity of ~2000 m/s at a stress of 2 GPa. Qualitatively, this behavior is fully consistent with the expected phonon-drag controlled motion of edge dislocations[14] confirmed earlier in MD simulations of BCC metals[18,40].

Of particular interest are velocities predicted from our learned mobility function for pure screw dislocations. When shear stress is applied in the twinning (T) direction (corresponding to a MRSSP angle $\chi = -30^o$), screw velocity reveals a well-defined threshold stress behavior characteristic of thermally-activated motion controlled by kink-pair mechanisms, with very low velocity at stresses below an activation threshold (finite temperature Peierls stress) followed by a transition to a drag-controlled regime at higher stresses. In contrast, when stress is applied in the anti-twinning (AT) direction (corresponding to a MRSSP angle $\chi = +30^o$), the same screw dislocation is predicted to move at barely discernible velocities which is again fully consistent with theoretical models[41] and earlier large-scale MD simulations[10]. Considering that a screw dislocation in a BCC crystal has a choice of three {112} glide planes at least one of which seeing resolved shear stress no less than $\sqrt{3}/2$ of shear stress resolved in the MRSSP, Fig. 2b indicates that, under conditions of our MD simulations and for the IAP used in this work, glide of screw dislocations on {112} planes in the AT directions is essentially prohibited.

DDD+ML simulations with learned mobility

To fully assess validity of our DDD+ML approach, we implemented our learned GNN dislocation mobility function in the ParaDiS DDD code[7] via the C++ PyTorch interface. For a one-to-one comparison, DDD simulations were performed under conditions identical to those used in the ground-truth MD simulations. Cube-shaped simulation boxes with the side length $296b$ (here $b = 0.2743$ nm is the magnitude of the Burgers vector), were initially seeded with randomly positioned prismatic loops and then deformed under tension and compression along the [001] axis at a strain rate of 2×10^8/s. For consistency, the shear modulus $\mu = 149.78$ GPa, the Poisson ratio $\nu = 0.289$ and the dislocation core terms[42] have been all computed at 300 K using the same IAP as in MD simulations[38].

As shown in Fig. 3, DDD predictions for flow stress agree closely with the ground-truth MD simulations. In particular, the asymmetry in flow stress under tension and under compression is fully captured in the learned GNN mobility function. However, in comparison to MD, DDD underpredicts dislocation densities (Fig. 3b) both under tension and under compression, even if asymmetry of two densities predicted in DDD is approximately the same as in MD. Potential sources of these discrepancies will be discussed in the next section.

To further test accuracy of our learned GNN mobility function, we now compare DDD and MD simulations performed under conditions never seen in training. For this purpose, we ran DDD and MD simulations under compression at a lower rate of 2×10^7/s, an order of magnitude lower than before. To ensure that simulations at the lower rate remain statically representative of bulk crystal plasticity, crystals used in the MD simulations were enlarged by a factor of four to ~140 millions atoms with a corresponding increase in the size of DDD simulation box to $470b$. Stress-strain curves extracted from MD and DDD simulations are plotted side by side in Fig. 4. Although our GNN dislocation mobility function was trained only on MD simulations performed under high deformation rate of 2×10^8/s, the plastic flow response predicted in the DDD+ML also agrees quantitatively with the MD simulation performed at the previously unseen lower rate of deformation.

This important observation is consistent with our earlier simulations based on a simplified DDD system[25] in which we similarly observed that a GNN model trained only at some deformation rate performs well in simulations performed at lower strain rates not seen in training. To rationalize this observed transferability of GNN mobility function to DDD simulations at lower deformation rates we note that, although the external/average flow stress stays nearly constant in our

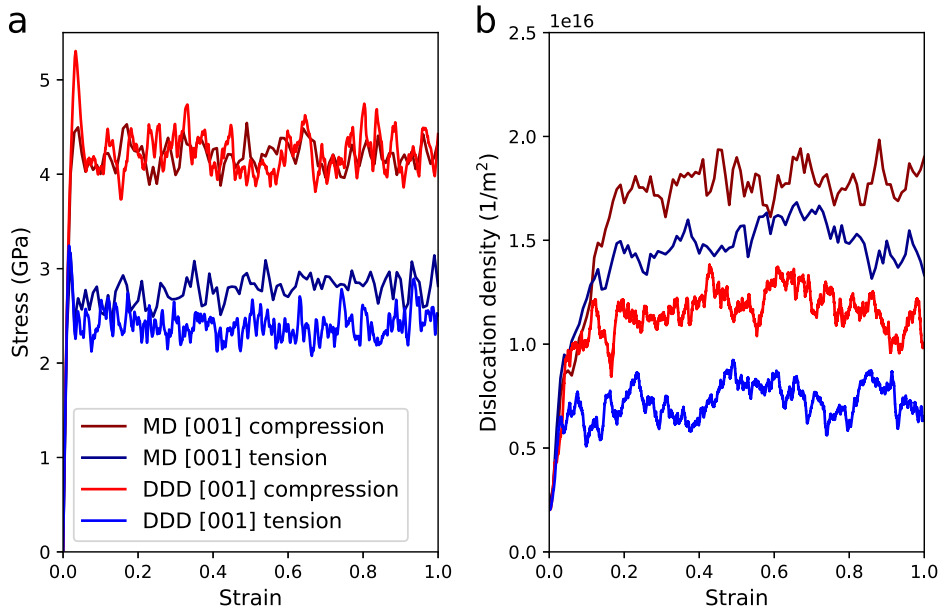

Fig. 3 | Large-scale DDD+ML predictions versus ground-truth MD results. a Comparison of stress-strain response predicted in MD and DDD simulations of [001] tension and compression deformation of BCC tungsten: the GNN dislocation mobility function employed in the DDD simulation was trained on the MD trajectories. MD and DDD simulations are performed in simulation volumes of the same size deformed under the same uniaxial straining conditions at rate 2×10^8/s. **b** Comparison of dislocation density evolution in the same MD and DDD simulations. Lines colors are the same as in the legend in (**a**).

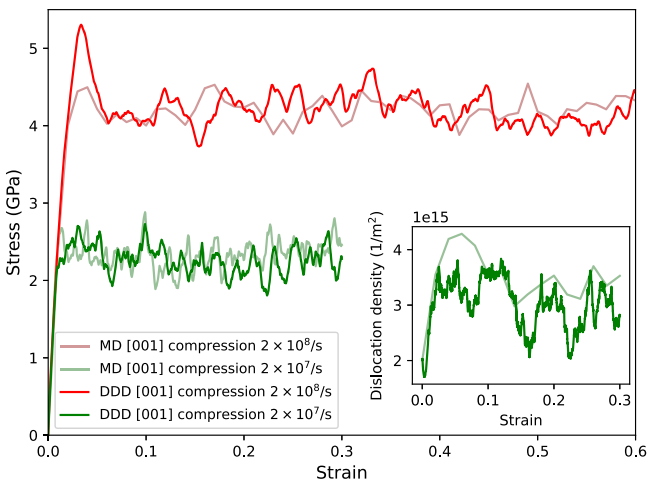

Fig. 4 | Comparison of stress-strain response predicted in MD and DDD simulations of [001] compression at rates 2×10^8/s and 2×10^7/s. Although the GNN mobility function was trained only on 2×10^8/s trajectories (red), it correctly predicts the flow stress at the lower straining rate of 2×10^7/s (green). Evolution of dislocation densities predicted at the lower straining rate is shown in the inset.

ground-truth training MD simulations, in the course of the same simulations dislocations experience a much wider range of stress conditions. Indeed, additional analysis of stress variations in simulation volumes of our MD training set shows that local stress (sum of average external stress and internal stress due to elastic interactions among dislocations) varies widely, from well above to well below the average stress. Accordingly, mechanical forces seen at any given time by many dislocations in the network can be low, as manifested in our previously documented observation that a large faction of dislocation segments move little or not at all from one DXA snapshot to the next[10]. Instances when mechanical driving force on a dislocation is low become increasingly common with decreasing deformation rates, but even at the high rates of our training MD simulations they occur sufficiently frequently for the GNN to learn on. We leave it to future work to further examine accuracy and transferability of our ML-learned dislocation mobility function across a wider range of deformation conditions.

Discussion

Results presented in the preceding sections demonstrate the feasibility and accuracy of our proposed DDD+ML approach. The same results are all the more significant in comparison to traditional "hand-crafted" mobility functions, including the ones previously developed in our group, that often fail to satisfactorily capture salient characteristics of crystal plasticity such as the tension/compression asymmetry or the peculiar slip crystallography in BCC crystals. Not commonly noted in the literature, such failures come in full display when comparing DDD predictions to predictions of MD simulations performed under identical conditions, which is precisely the essence of a cross-scale matching approach[24] we presently pursue. Here we take advantage of a large number of instances of dislocation motion encountered in large-scale MD simulations of crystal plasticity to train a machine learner to advance dislocations forward in time in a DDD simulation in the same manner they move in MD. While fully accounting for known physics in a data-driven fashion, the GNN learns on local network configurations to predict corrections to dislocation motion that are otherwise difficult if not impossible to evaluate. Once incorporated in a DDD model, our learned mobility function accurately describes complex motion of dislocations in a variety of local configurations and stress states resulting in strongly anisotropic plasticity response of BCC metals[33,34,43,44]. Through observing a large number of examples during training, the GNN also learns to predict motion of isolated dislocations for which it predicts smooth stress-velocity relations in close agreement with existing theory, e.g., see Fig. 2b.

Of particular interest is that, once trained, our GNN mobility function encodes information about slip crystallography without any prior assumption about planes in which dislocations should prefer to glide, a still contentious issue in BCC metal plasticity[37]. This is in contrast to previous work in which a fixed set of glide planes and a certain functional form of the dislocation mobility function are assumed a priori[14,16]. At present when physics of dislocation motion is not yet fully understood, imposing any such a priori constraints may prove detrimental to fidelity of DDD simulations.

In our first application of the DDD+ML approach, we nevertheless observe discrepancies in DDD predictions compared to the ground-truth MD simulations. While under compression DDD and MD predictions for flow stress agree within statistical uncertainty, flow stress under tension is slightly under-predicted in DDD. Yet the most notable discrepancy is found in the evolution of dislocation densities that are under-predicted in DDD by about 50% both under tension and under compression. Additional work is required to establish what causes these discrepancies, but several sources appear plausible and worth further examination.

One plausible source of discrepancies in dislocation multiplication is that in DDD dislocation networks evolve not only via the motion of dislocation nodes or segments, but also through a sequence of topological transformations, e.g., dislocation collisions, junction zipping and unzipping and such. Although taking place both in MD and DDD, DDD models are never really informed how such topological events take place in MD but rely instead on plausible and yet to be validated theoretical arguments such as maximum power dissipation[7]. Furthermore, in mapping dislocation line networks on a regular grid and comparing them on the basis of their continuous Nye's field representations, as was done in training the GNN mobility function, any topological events taking place between two consecutive DXA snapshots are integrated over and lost. Thus, strictly speaking, the GNN only learns how to advance the Nye's field on the grid while never learning to perform topological transformations in the dislocation line network the way they take place in MD. We suspect that, in not matching MD on topological events, DDD can not match MD on dislocation multiplication. Exactly where and how dislocations multiply in a massive MD simulation remains an important open question for future studies.

We also note that the resulting dislocation behaviors captured in our trained GNN mobility is naturally tied to the choice of IAP used to generate the ground-truth MD data. For instance, Finnis-Sinclair style potentials such as that used to model W in this work[38] are known to predict degenerate screw core structures[44], which may affect predictions of slip crystallography in the ground-truth MD trajectories. Yet, we emphasize that our primary objective here is to develop a generic framework capable of accurately capturing details of dislocation motion as seen in the ground-truth data, agnostic to the quality of the ground-truth data itself. Should a more accurate IAP be employed for generating training data, our DDD+ML workflow should be able to learn the resulting dislocation motion equally well. It is also possible that advanced techniques of transfer-learning may be used in the future to fine-tune GNN mobilities initially trained on "cheap" IAPs to learn dislocation behaviors from higher-fidelity IAPs, requiring only minimal additional training data. Furthermore, coupling of the model with a physics-informed neural networks (PINN)[45] approach may provide a path to improve the accuracy and generalizability of the proposed approach in the future.

As a coarse-grained model, DDD simulations are typically orders of magnitude faster to perform than MD simulations under identical conditions. Although requiring more calculations than the traditional hand-crafted mobility functions, our trained GNN mobility function still comes at a negligible computational cost compared to force calculations. Considering simulations reported in Fig. 3 (at 2×10^8/s), an MD simulation requires $\sim 10,000$ CPU-hours (using a cheap EAM IAP), compared to $\sim 10-20$ CPU-hours needed to complete a DDD simulation performed under identical conditions. The speedup of DDD over MD is only going to grow larger with more sophisticated IAPs and decreasing deformation rates where, in addition to its considerably longer time steps, DDD remains efficient in much larger simulation volumes (e.g. several μm) that are presently out-of-reach for MD simulations.

Finally, as a first application here we demonstrate our ML-DDD framework on BCC crystals in which dislocation cores are compact. To extend our approach to FCC and HCP crystals where dislocations dissociate into partials, we envision two possible paths depending on the material. For crystals with high stacking-fault energy (SFE), a possible approach would be to constrain DXA to extract only the complete dislocations, learn a mobility function on the resulting DXA snapshots and use it in subsequent DDD simulations of the same material. For low SFE crystals in which two partials may move far apart, pairing the partials into complete dislocations may not be physically justifiable or even possible. Here a more suitable approach would be to learn mobility of the partial dislocations and use it in a DDD model resolving the partials[46]. Extension of the framework to more complex materials and material microstructures (e.g. irradiated materials, impurities, etc.) should also be possible in the future, e.g., by conditioning the local GNN mobility function on the presence of other types of crystal defects.

In summary, we propose a data centered approach to constructing DDD mobility functions in which a GNN model is trained on large-scale MD data. Training is achieved by minimizing DDD prediction loss for the next state of dislocation network conditioned on mechanical forces computed for the previous state using standard DDD equations. To circumvent uncertainties in comparing generally non-isomorphic graphs, dislocation networks are mapped on a regular grid using a continuous Nye's tensor field representation. To test our new approach on a complex case of crystal plasticity in BCC tungsten, we ran DDD simulations using a GNN dislocation mobility function trained on ground-truth MD simulations. The resulting DDD predictions match MD simulation used in training as well as MD simulations performed under previously unseen deformation conditions. We believe our data-driven approach to constructing dislocation mobility functions is a promising avenue for improving prediction fidelity of the DDD method.

Methods

GNN mobility law

We model the mobility law in Eq. (4) with a message-passing GNN[47], which has proved very powerful for predicting force fields[48–51], defect-properties linkage[52], fracture mechanics[53], polycrystal properties[54,55], and for simulating complex physics[56].

Following our recent work on applying GNN to a simplified model of DDD simulations[25], a dislocation configuration is represented by a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V} = \{N_i\}$ is a collection of dislocation node/vertex features (attributes), and $\mathcal{E} = \{E_{ij}\}$ is a collection of dislocation segment/edge features. We define the input features for each node i as

$$N_i = \left(n_i, \boldsymbol{F}^{\mathrm{DDD}}, \| \boldsymbol{F}^{\mathrm{DDD}} \|\right) \tag{7}$$

where n_i is a flag used to specify whether node i is a discretization or physical (junction) node. The input edge features on edge ij are

$$E_{ij} = (e_{ij}, \boldsymbol{b}_{ij}, \| \boldsymbol{b}_{ij} \|, \boldsymbol{r}_j - \boldsymbol{r}_i, \| \boldsymbol{r}_j - \boldsymbol{r}_i \|) \tag{8}$$

where e_{ij} is a flag used to specify whether segment ij is a glissile or junction segment, and $\boldsymbol{r}_j - \boldsymbol{r}_i$ are the local segments line vectors, naturally compatible with the use of periodic boundary conditions. To satisfy Burgers vector conservation, we use a directed graph, i.e. if ij is an edge with Burgers vector $\boldsymbol{b}_{ij}$, then ji is also an edge but with opposite Burgers vector $-\boldsymbol{b}_{ij}$.

Our GNN architecture follows[57] and is first composed of vertex and edge encoders ENC^V, ENC^E transforming concatenated input features into a latent space

$$v_i^{(0)} = \mathrm{ENC}^V(N_i),\ e_{ij}^{(0)} = \mathrm{ENC}^E(E_{ij}), \tag{9}$$

followed by K stacked message passing layers $f^{E(k)}, f^{V(k)}$ $(1 \le k \le K)$ sequentially updating the latent vertex and edge variables

$$e_{ij}^{(k)} = f^{E(k)}(e_{ij}^{(k-1)}, v_i^{(k-1)}, v_j^{(k-1)}), \tag{10}$$

$$v_i^{(k)} = f^{V(k)}\left(v_i^{(k-1)}, \sum_j e_{ij}^{(k)}\right), \tag{11}$$

and finally a node decoder DEC that translates the latent node variables $v^{(K)}$ into the desired output properties, i.e. nodal velocity vectors:

$$\boldsymbol{V}_i = \mathrm{DEC}\left(\mathrm{v}_\mathrm{i}^{(\mathrm{K})}\right). \tag{12}$$

Functions ENC^V, ENC^E, f^V, f^E, and DEC are neural network operators built from multi-layer perceptrons with two hidden layers, layer normalization[58], skip connections[59], and GELU activation functions[60].

MD simulations

Large-scale MD simulations of BCC tungsten are performed with LAMMPS[61] under the Kokkos GPU implementation[62], using the EAM-style IAP developed in ref. 38. Simulations are performed following the protocol introduced in ref. 8. Periodic, orthorombic BCC perfect crystals are initially seeded with twelve $1/2\langle 111\rangle\{110\}$ prismatic loops of the vacancy type. The crystals are first equilibrated at the temperature of 300 K, after which they are deformed at a constant true strain rate. Temperature and uniaxial loading conditions are maintained during deformation using the *langevin* thermostat and the *nph* barostat. For the ~ 35 millions atoms simulations deformed at a rate of 2×10^8/s, DXA[27] is executed every 1 ps to save the detailed evolution of the dislocation networks.

Mobility workflow and training

DXA configurations produced in the MD simulations are first converted to the ParaDiS format[7]. For consistency, during this operation the line networks are also remeshed with discretization size of ~ $10b$, corresponding to the average segment size used in our DDD simulations.

The so-converted dislocation configurations $\{\mathcal{G}^{t_s}\}$ are then fed to the ParaDiS code to compute nodal forces, Eq. (2). Applied forces $\boldsymbol{F}_i^{app}$ are computed by integrating the Peach-Koehler force $(\boldsymbol{\sigma}^{t_s} \cdot \boldsymbol{b}_{ij}) \times \boldsymbol{t}_{ij}$ along the dislocation segments ij with unit tangent $\boldsymbol{t}_{ij}$, where $\boldsymbol{\sigma}^{t_s}$ is the instantaneous stress applied to network $\mathcal{G}^{t_s}$ at time t_s as recorded during the MD runs. Long-range interaction forces $\boldsymbol{F}_i^{lr}$ are computed using DDD-FFT approach introduced in ref. 29 which can easily handle non-cubic, deforming simulation boxes as produced by MD simulations. Short-range interaction forces $\boldsymbol{F}_i^{sr}$ are computed using the non-singular isotropic analytical formulation[63]. Core forces $\boldsymbol{F}_i^{core}$ are computed from core energies extracted from the ground-truth IAP[38] using the framework developed in ref. 42.

The networks containing nodal forces are then used as inputs to our training procedure implemented within PyTorch[30]. To facilitate training, input forces are rescaled by a factor $10^9\text{Pa} \cdot b^2$ so that their average magnitude is on the order of unity. The loss function, Eq. (5), is computed by evaluating the Nye's tensor on a grid of 32^3 voxels using a fully vectorized implementation of the discrete-to-continuous method introduced in ref. 29.

We trained different GNN models to explore different sets of hyperparameters. We tested a combination of models with $K = \{2, 3\}$ message-passing layers each with latent space of size $L = \{48, 96\}$, leading to 4 different trained models. The models were trained using the AdamW optimizer with weight decay of 1×10^{-5}[64]. Training was performed for 12 hours with batch size of 4 on a single NVidia V100 GPU. We find that the GNN model with $K = 3$ and $L = 48$ offers the best trade-off between accuracy and complexity (78,768 total parameters) while showing no sign of ovefitting. We thus selected it as the best model for results presented in this work.

Data availability

The datasets used and analysed during the current study are available from the corresponding author on reasonable request.

Code availability

The code used in the current study will be made available in an open-source repository.

Received: 25 March 2024; Accepted: 4 August 2024;
Published online: 27 August 2024

References

1. Kubin, L. P. et al. Dislocation microstructures and plastic flow: a 3D simulation. *Solid state Phenom.* **23**, 455–472 (1992).
2. Zbib, H. M., Rhee, M. & Hirth, J. P. On plastic deformation and the dynamics of 3D dislocations. *Int. J. Mech. Sci.* **40**, 113–127 (1998).
3. Bulatov, V., Abraham, F. F., Kubin, L., Devincre, B. & Yip, S. Connecting atomistic and mesoscale simulations of crystal plasticity. *Nature* **391**, 669–672 (1998).
4. Schwarz, K. Simulation of dislocations on the mesoscopic scale. i. methods and examples. *J. Appl. Phys.* **85**, 108–119 (1999).
5. Ghoniem, N. A. M., Tong, S.-H. & Sun, L. Parametric dislocation dynamics: a thermodynamics-based approach to investigations of mesoscopic plastic deformation. *Phys. Rev. B* **61**, 913 (2000).
6. Weygand, D., Friedman, L., Van der Giessen, E. & Needleman, A. Aspects of boundary-value problem solutions with three-dimensional dislocation dynamics. *Model. Simul. Mater. Sci. Eng.* **10**, 437 (2002).
7. Arsenlis, A. et al. Enabling strain hardening simulations with dislocation dynamics. *Model. Simul. Mater. Sci. Eng.* **15**, 553–595 (2007).
8. Zepeda-Ruiz, L. A., Stukowski, A., Oppelstrup, T. & Bulatov, V. V. Probing the limits of metal plasticity with molecular dynamics simulations. *Nature* **550**, 492–495 (2017).
9. Zepeda-Ruiz, L. A. et al. Atomistic insights into metal hardening. *Nat. Mater.* **20**, 315–320 (2021).
10. Bertin, N., Zepeda-Ruiz, L. & Bulatov, V. Sweep-tracing algorithm: in silico slip crystallography and tension-compression asymmetry in bcc metals. *Mater. Theory* **6**, 1–23 (2022).
11. Stimac, J. C., Bertin, N., Mason, J. K. & Bulatov, V. V. Energy storage under high-rate compression of single crystal tantalum. *Acta Materialia* **239**, 118253 (2022).
12. Monnet, G. & Terentyev, D. Structure and mobility of the $12\langle 111\rangle\{112\}$ edge dislocation in bcc iron studied by molecular dynamics. *Acta Materialia* **57**, 1416–1426 (2009).
13. Kang, K., Bulatov, V. V. & Cai, W. Singular orientations and faceted motion of dislocations in body-centered cubic crystals. *Proc. Natl Acad. Sci.* **109**, 15174–15178 (2012).
14. Po, G. et al. A phenomenological dislocation mobility law for bcc metals. *Acta Materialia* **119**, 123–135 (2016).
15. Geslin, P.-A. & Rodney, D. Thermal fluctuations of dislocations reveal the interplay between their core energy and long-range elasticity. *Phys. Rev. B* **98**, 174115 (2018).
16. Wang, Z. & Beyerlein, I. An atomistically-informed dislocation dynamics model for the plastic anisotropy and tension–compression asymmetry of bcc metals. *Int. J. Plasticity* **27**, 1471–1484 (2011).
17. Srivastava, K., Gröger, R., Weygand, D. & Gumbsch, P. Dislocation motion in tungsten: atomistic input to discrete dislocation simulations. *Int. J. Plasticity* **47**, 126–142 (2013).
18. Chang, J., Cai, W., Bulatov, V. V. & Yip, S. Dislocation motion in bcc metals by molecular dynamics. *Mater. Sci. Eng.: A* **309**, 160–163 (2001).
19. Olmsted, D. L., Hector, L. G., Curtin, W. & Clifton, R. Atomistic simulations of dislocation mobility in Al, Ni and Al/Mg alloys. *Model. Simul. Mater. Sci. Eng.* **13**, 371 (2005).
20. Queyreau, S., Marian, J., Gilbert, M. & Wirth, B. Edge dislocation mobilities in bcc Fe obtained by molecular dynamics. *Phys. Rev. B* **84**, 064106 (2011).
21. Cereceda, D. et al. Assessment of interatomic potentials for atomistic analysis of static and dynamic properties of screw dislocations in W. *J. Phys.: Condens. Matter* **25**, 085702 (2013).
22. Cho, J., Molinari, J.-F. & Anciaux, G. Mobility law of dislocations with several character angles and temperatures in fcc aluminum. *Int. J. Plasticity* **90**, 66–75 (2017).
23. Bertin, N., Sills, R. B. & Cai, W. Frontiers in the simulation of dislocations. *Annu. Rev. Mater. Res.* **50**, 437–464 (2020).
24. Bertin, N., Cai, W., Aubry, S., Arsenlis, A. & Bulatov, V. V. Enhanced mobility of dislocation network nodes and its effect on dislocation multiplication and strain hardening. *Acta Materialia* **271**, 119884 (2024).
25. Bertin, N. & Zhou, F. Accelerating discrete dislocation dynamics simulations with graph neural networks. *J. Comput. Phys.* **487**, 112180 (2023).
26. Stukowski, A. & Albe, K. Extracting dislocations and non-dislocation crystal defects from atomistic simulation data. *Model. Simul. Mater. Sci. Eng.* **18**, 085001 (2010).

27. Stukowski, A. A triangulation-based method to identify dislocations in atomistic models. *J. Mech. Phys. Solids* **70**, 314–319 (2014).
28. Nye, J. Some geometrical relations in dislocated crystals. *Acta Metall.* **1**, 153–162 (1953).
29. Bertin, N. Connecting discrete and continuum dislocation mechanics: A non-singular spectral framework. *Int. J. Plasticity* **122**, 268–284 (2019).
30. Paszke, A. et al. Pytorch: An imperative style, high-performance deep learning library, in: Advances in Neural Information Processing Systems 32, Curran Associates, Inc., 2019, pp. 8024–8035.
31. Cai, W. & Bulatov, V. V. Mobility laws in dislocation dynamics simulations. *Mater. Sci. Eng.: A* **387**, 277–281 (2004).
32. Christian, J. Some surprising features of the plastic deformation of body-centered cubic metals and alloys. *Metall. Trans. A* **14**, 1237–1256 (1983).
33. Duesbery, Ma-S. & Vitek, V. Plastic anisotropy in bcc transition metals. *Acta Materialia* **46**, 1481–1492 (1998).
34. Dezerald, L., Rodney, D., Clouet, E., Ventelon, L. & Willaime, F. Plastic anisotropy and dislocation trajectory in bcc metals. *Nat. Commun.* **7**, 11695 (2016).
35. Sherwood, P., Guiu, F., Kim, H. C. & Pratt, P. L. Plastic anisotropy of tantalum, niobium, and molybdenum. *Can. J. Phys.* **45**, 1075–1089 (1967).
36. Webb, G. L., Gibala, R. & Mitchell, T. E. Effect of normal stress on yield asymmetry in high purity tantalum crystals. *Metall. Trans.* **5**, 1581–1584 (1974).
37. Weinberger, C. R., Boyce, B. L. & Battaile, C. C. Slip planes in bcc transition metals. *Int. Mater. Rev.* **58**, 296–314 (2013).
38. Juslin, N. & Wirth, B. Interatomic potentials for simulation of he bubble formation in W. *J. Nucl. Mater.* **432**, 61–66 (2013).
39. Bertin, N., Carson, R., Bulatov, V. V., Lind, J. & Nelms, M. Crystal plasticity model of bcc metals from large-scale MD simulations. *Acta Materialia* **260**, 119336 (2023).
40. Osetsky, Y. N. & Bacon, D. J. An atomic-level model for studying the dynamics of edge dislocations in metals. *Model. Simul. Mater. Sci. Eng.* **11**, 427 (2003).
41. Edagawa, K., Suzuki, T. & Takeuchi, S. Motion of a screw dislocation in a two-dimensional Peierls potential. *Phys. Rev. B* **55**, 6180 (1997).
42. Bertin, N., Cai, W., Aubry, S. & Bulatov, V. Core energies of dislocations in bcc metals. *Phys. Rev. Mater.* **5**, 025002 (2021).
43. Ito, K. & Vitek, V. Atomistic study of non-Schmid effects in the plastic yielding of bcc metals. *Philos. Mag. A* **81**, 1387–1407 (2001).
44. Vitek, V. Core structure of screw dislocations in body-centred cubic metals: relation to symmetry and interatomic bonding. *Philos. Mag.* **84**, 415–428 (2004).
45. Raissi, M., Perdikaris, P. & Karniadakis, G. E. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *J. Comput. Phys.* **378**, 686–707 (2019).
46. Martinez, E., Marian, J., Arsenlis, A., Victoria, M. & Perlado, J. M. Atomistically informed dislocation dynamics in fcc crystals. *J. Mech. Phys. Solids* **56**, 869–895 (2008).
47. Battaglia, P. W. et al. Relational inductive biases, deep learning, and graph networks. http://arxiv.org/abs/1806.01261.(2018).
48. Gilmer, J., Schoenholz, S. S., Riley., P. F., Vinyals, O., Dahl, G. E. Neural message passing for quantum chemistry, in: International conference on machine learning, PMLR, 2017, pp. 1263–1272.
49. Chen, C., Ye, W., Zuo, Y., Zheng, C. & Ong, S. P. Graph Networks as a Universal Machine Learning Framework for Molecules and Crystals. *Chem. Mater.* **31**, 3564 (2018).
50. Park, C. W. et al. Accurate and scalable graph neural network force field and molecular dynamics with direct force architecture. *npj Comput. Mater.* **7**, 73 (2021).
51. Batzner, S. et al. E(3)-equivariant graph neural networks for data-efficient and accurate interatomic potentials. *Nat. Commun.* **13**, 2453 (2022).
52. Yang, Z. & Buehler, M. J. Linking atomic structural defects to mesoscale properties in crystalline solids using graph neural networks. *Npj Comput. Mater.* **8**, 198 (2022).
53. Perera, R., Guzzetti, D. & Agrawal, V. Graph neural networks for simulating crack coalescence and propagation in brittle materials. *Computer Methods Appl. Mech. Eng.* **395**, 115021 (2022).
54. Dai, M., Demirel, M. F., Liang, Y. & Hu, J.-M. Graph neural networks for an accurate and interpretable prediction of the properties of polycrystalline materials. *npj Comput. Mater.* **7**, 103 (2021).
55. Hestroffer, J. M., Charpagne, M.-A., Latypov, M. I. & Beyerlein, I. J. Graph neural networks for efficient learning of mechanical properties of polycrystals. *Comput. Mater. Sci.* **217**, 111894 (2023).
56. Sanchez-Gonzalez, A. et al. Learning to simulate complex physics with graph networks, in: International Conference on Machine Learning, PMLR, 2020, pp. 8459–8468.
57. Pfaff, T., Fortunato, M., Sanchez-Gonzalez, A., Battaglia, P. W. Learning mesh-based simulation with graph networks. http://arxiv.org/abs/2010.03409 (2020).
58. Ba, J. L., Kiros, J. R., Hinton, G. E. Layer normalization. http://arxiv.org/abs/1607.06450 (2016).
59. He, K., Zhang, X., Ren, S., Sun, J. Deep residual learning for image recognition. http://arxiv.org/abs/1512.03385 (2015).
60. Hendrycks, D., Gimpel, K. Gaussian error linear units (GELUs). http://arxiv.org/abs/1606.08415 (2016).
61. Thompson, A. P. et al. Lammps-a flexible simulation tool for particle-based materials modeling at the atomic, meso, and, continuum scales. *Computer Phys. Commun.* **271**, 108171 (2022).
62. Edwards, H. C., Trott, C. R. & Sunderland, D. Kokkos: Enabling manycore performance portability through polymorphic memory access patterns. *J. Parallel Distrib. Comput.* **74**, 3202–3216 (2014).
63. Cai, W., Arsenlis, A., Weinberger, C. R. & Bulatov, V. V. A non-singular continuum theory of dislocations. *J. Mech. Phys. Solids* **54**, 561–587 (2006).
64. Loshchilov, I., Hutter, F. Decoupled weight decay regularization. http://arxiv.org/abs/1711.05101 (2017).

Acknowledgements

NB and VB acknowledge support by the Laboratory Directed Research and Development (LDRD) program (22-ERD-016) and by the ASC PEM program at Lawrence Livermore National Laboratory (LLNL). FZ was supported by the Critical Materials Innovation Hub, an Energy Innovation Hub funded by the U.S. Department of Energy, Office of Energy Efficiency and Renewable Energy, and Advanced Materials and Manufacturing Technologies Office. Computing support for this work came from LLNL Institutional Computing Grand Challenge program. This work was performed under the auspices of the U.S. Department of Energy by LLNL under contract DE-AC52-07NA27344.

Author contributions

N.B. developed the method, performed the simulations, and trained the model. N.B. and F.Z. implemented the model. N.B. and V.B. analyzed the results and wrote the manuscript. All authors read and approved the final manuscript.

Competing interests

The authors declare no competing interests.

Additional information

Correspondence and requests for materials should be addressed to Nicolas Bertin.

Reprints and permissions information is available at http://www.nature.com/reprints

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

© The Author(s) 2024